GUI应用的生命周期与资源管理

1. 课程目标

写出一个能跑的 App 很简单，但写出一个**稳定**、**不泄露内存**的 App 是需要多加注意的。本节课我们将深入讲解 App 的生命周期管理。

2. 生命周期状态

一个 App 在运行过程中会经历以下状态：

1. **加载 (Load)**: 系统启动，`AppManager` 扫描到 `app.py`，执行 `__init__`。此时 App 处于休眠状态，占用极少内存。
2. **激活 (Launch)**: 用户点击图标，执行 `launch_with_animation` -> `initialize`。此时 UI 资源被创建，占用内存增加。
3. **运行 (Running)**: 用户与界面交互。
4. **暂停/后台 (Suspend)**: 在 Yahboom K230 的 GUI 中，目前主要支持单任务前台运行，所谓的"后台"则意味着被销毁。
5. **销毁 (Destroy)**: 用户点击返回键，App 退出，释放资源。

3. 正确的资源释放

在 `BaseApp` 中，虽然没有显式的 `destroy` 方法，但当 `launch_with_animation` 创建的 `self.screen` 被删除时，挂载在上面的所有控件都会被自动删除。

但是，有一些资源是 LVGL **管不到**的，必须手动释放：

3.1 定时器 (Timers)

如果你在 App 里创建了定时器：

```
self.timer = Timer(1)
self.timer.init(period=1000, mode=Timer.PERIODIC, callback=self.update)
```

必须在退出时关闭它，否则即便 App 关闭了，定时器还在后台跑，最终导致报错或系统卡死。

最佳实践:
重写 `on_close` 方法：

```
    def on_deinit(self):
        if self.timer:
            self.timer.deinit()
```

3.2 大图片缓存

如果你加载了巨大的图片到内存中 (`f.read()`)，退出时建议手动调用垃圾回收：

```
import gc

def on_deinit(self):
    self.big_image_data = None
    gc.collect() # 强制回收内存
```